

Efeitos temporários

Nem todo bônus deveria durar para sempre. Hoje um bônus aprende a contar os próprios turnos e a ir embora sozinho.

NESTA AULA

1. Recapitular onde vocês pararam
2. Construir `aplicar_bonus_ataque_temporario`
3. Ensinar `iniciar_turno` a descontar o bônus quando ele vence
4. Construir o Elixir de fúria, o primeiro item que usa o padrão novo
5. Colocar o elixir como prêmio de fase

Antes de começar, confira a sua pasta:

ARQUIVO	O QUE TEM DENTRO
<code>personagem.py</code>	Personagem, <code>nivel</code> , <code>subir_nivel</code> , <code>melhorar_ataque</code> / <code>defesa</code> / <code>vida_maxima</code>
<code>itens.py</code>	Item, PocaoDeVida, Bomba, PocaoDeForca, AnelDeProtecao, Inventario
<code>classes.py</code>	Guerreiro, Mago, Goblin, Orc, Troll, Dragao
<code>main.py</code>	Fase, a lista <code>fases</code> , o menu, o combate, <code>subir_nivel</code> chamado a cada vitória

ESTE DOCUMENTO É PARA SER EXPLICADO

Assim como no material de mais itens, este PDF mostra o código já pronto — ele não é para o aluno seguir sozinho copiando. Os exercícios, sem o código dado, estão num segundo documento.

Onde vocês pararam

O personagem agora melhora sozinho a cada vitória. `subir_nivel` chama três métodos que só sabem **somar** — nenhum deles sabe subtrair de volta:

```
def melhorar_ataque(self, quantidade):
    self.ataque += quantidade

def subir_nivel(self):
    self.nivel += 1
    self.melhorar_ataque(4)
    self.melhorar_defesa(2)
    self.melhorar_vida_maxima(20)
    print(self.nome, "subiu para o nível", self.nivel)
```

Isso está certo para nível — ninguém quer perder o nível depois de um tempo. Mas nem todo bônus é assim. No fim da aula passada, ficou esta pergunta, dentro de um método que vocês já conhecem:

```
def iniciar_turno(self):
    self.defendendo = False
    # ... e se um bônus tivesse que "acabar" aqui?
```

`iniciar_turno` já reseta `defendendo` no começo de cada turno. Ele é, sem vocês perceberem ainda, o lugar exato onde qualquer coisa "temporária" no personagem deveria ser conferida. Hoje ele ganha um segundo trabalho.

VOCABULÁRIO RÁPIDO, SE ALGUÉM ESQUECEU

`self.ataque` — atributo comum, qualquer método da classe muda ele direto.

`iniciar_turno` — chamado uma vez no começo do turno de cada personagem, dentro do laço de `combate`, antes de qualquer ação.

bônus temporário (novo hoje) — um valor que soma em um atributo por um número certo de turnos, e depois desfaz sozinho a própria soma.

aplicar_bonus_ataque_temporario

Um bônus para sempre só precisa lembrar de uma coisa: o quanto somar. Um bônus temporário precisa lembrar de duas: o quanto, e por quantos turnos ainda vale.

O QUE ELE PRECISA FAZER

- Somar `quantidade` em `self.ataque`, imediatamente — igual a um bônus comum.
- Guardar `quantidade` em `self._bonus_ataque`, para lembrar quanto desfazer depois.
- Guardar `turnos` em `self._turnos_bonus_ataque`, para saber quando desfazer.

E dois atributos novos no `__init__`, começando zerados:

```
class Personagem:
    def __init__(self, nome, vida, ataque, defesa, pocoes):
        ...
        self.nivel = 1
        self._bonus_ataque = 0
        self._turnos_bonus_ataque = 0
        ...

    def aplicar_bonus_ataque_temporario(self, quantidade, turnos):
        self.ataque += quantidade
        self._bonus_ataque = quantidade
        self._turnos_bonus_ataque = turnos
        print(self.nome, "sente o ataque aumentar por um tempo")
```

QUANTO CÓDIGO ISSO É

Cinco linhas no método, duas no `__init__`. Mas repare que ele ainda não desfaz nada sozinho — só aplica e lembra. Quem desfaz é o próximo método.

Bônus para sempre, bônus com prazo

melhorar_ataque (ontem)

```
def melhorar_ataque(  
    self, quantidade):  
    self.ataque += quantidade
```

aplicar_bonus_ataque_temporario (hoje)

```
def aplicar_bonus_ataque_temporario(  
    self, quantidade, turnos):  
    self.ataque += quantidade  
    self._bonus_ataque = quantidade  
    self._turnos_bonus_ataque = turnos  
    print(self.nome,  
          "sente o ataque aumentar por um  
          tempo")
```

- ganha um segundo parâmetro
- ganha duas linhas que só lembram, não mudam nada ainda

A DIFERENÇA, E SÓ ESSA

A primeira linha das duas é idêntica: `self.ataque += quantidade`.

`aplicar_bonus_ataque_temporario` não faz a soma de um jeito diferente — ele só **também** escreve numa agenda o que vai precisar desfazer depois. Sem essa agenda, o bônus temporário seria, na prática, permanente: ninguém saberia quanto tirar de volta.

Testando

```
h = Guerreiro("Thoric")  
print("ataque:", h.ataque)  
h.aplicar_bonus_ataque_temporario(6, 3)  
print("ataque:", h.ataque)
```

SAÍDA

```
ataque: 18  
Thoric sente o ataque aumentar por um tempo  
ataque: 24
```

Duas coisas para lembrar, não uma

`quantidade` e `turnos` respondem perguntas diferentes: quanto tirar, e quando tirar. Guardar só uma das duas não é suficiente.

ERRO COMUM DE COPIAR E COLAR

Quem copia `melhorar_ataque` e só troca o nome do método, sem pensar no resto, escreve isto:

```
def aplicar_bonus_ataque_temporario(self, quantidade, turnos):  
    self.ataque += quantidade    # só isso
```

O programa não quebra. O bônus é aplicado, o teste do checkpoint anterior até passa. Mas `_bonus_ataque` e `_turnos_bonus_ataque` nunca são atualizados — quando o momento de desfazer chegar, não existe registro de quanto foi somado. Na prática, esse "bônus temporário" nunca vai embora. É um bug silencioso: nada avisa que ele existe até alguém notar que o personagem ficou forte demais.

VALE PENSAR

O que acontece se `aplicar_bonus_ataque_temporario` for chamado de novo enquanto um bônus anterior ainda está ativo? Do jeito que está escrito, o segundo `self._turnos_bonus_ataque = turnos` apaga a contagem antiga e começa outra — o personagem fica com o ataque somado duas vezes, mas só um prazo rodando. Hoje isso fica assim, de propósito: só existe um bônus de ataque por vez neste jogo.

Onde o bônus termina

`aplicar_bonus_ataque_temporario` aplica e lembra. Falta o método que confere a lembrança a cada turno e desfaz a soma quando o prazo acaba — e esse método já existe: é o `iniciar_turno` de sempre.

O QUE ELE PRECISA FAZER, ALÉM DO QUE JÁ FAZIA

- Se ainda houver turnos de bônus (`self._turnos_bonus_ataque > 0`), descontar um turno.
- Se, depois de descontar, os turnos chegarem a zero, subtrair `self._bonus_ataque` do `ataque` e zerar o registro.

```
def iniciar_turno(self):
    self.defendendo = False

    if self._turnos_bonus_ataque > 0:
        self._turnos_bonus_ataque -= 1
        if self._turnos_bonus_ataque == 0:
            self.ataque -= self._bonus_ataque
            self._bonus_ataque = 0
            print(self.nome, "sente o ataque voltar ao normal")
```

QUANTO CÓDIGO ISSO É

Cinco linhas novas, dentro de um método que já existia. Nenhum outro método do jogo precisa saber que bônus temporários existem — `combate` já chama `iniciar_turno` a cada turno, para os dois lados, há aulas.

O método de sempre, com um trabalho a mais

iniciar_turno (até ontem)

```
def iniciar_turno(self):
    self.defendendo = False
```

iniciar_turno (hoje)

```
def iniciar_turno(self):
    self.defendendo = False

    if self._turnos_bonus_ataque > 0:
        self._turnos_bonus_ataque -= 1
    if self._turnos_bonus_ataque == 0:
        self.ataque -= (
            self._bonus_ataque)
        self._bonus_ataque = 0
        print(
            self.nome,
            "sente o ataque"
            " voltar ao normal",
        )
```

- a linha de sempre não muda
- o bloco novo só age quando existe um bônus rodando

POR QUE DENTRO DE INICIAR_TURNO, E NÃO EM OUTRO LUGAR

`iniciar_turno` já é chamado automaticamente, no momento certo, para o jogador e para o inimigo, sem ninguém precisar lembrar de chamar nada a mais. Colocar a lógica do bônus ali significa que ela roda de graça, todo turno, para qualquer personagem — sem `combate` precisar saber que bônus temporários existem.

Testando

```
h = Guerreiro("Thoric")
h.aplicar_bonus_ataque_temporario(6, 3)
for _ in range(3):
    h.iniciar_turno()
    print("ataque:", h.ataque)
```

SAÍDA

```
Thoric sente o ataque aumentar por um tempo
ataque: 24
ataque: 24
Thoric sente o ataque voltar ao normal
ataque: 18
```

Contando exatamente três turnos

Reparem no teste da página anterior: `duracao=3`, mas o ataque volta ao normal só na **terceira** chamada de `iniciar_turno`, não na primeira. O turno em que o elixir é bebido já conta como o primeiro turno de bônus — beber e imediatamente perder o efeito no mesmo turno não faria sentido.

ERRO COMUM: SUBTRAIR TODO TURNO

Quem esquece o `if` de dentro e escreve só isto:

```
def iniciar_turno(self):
    self.defendendo = False
    self._turnos_bonus_ataque -= 1
    self.ataque -= self._bonus_ataque # roda sempre, nao so no fim
```

vai ver o ataque despencar todo turno, inclusive quando não existe bônus nenhum ativo — porque `self._bonus_ataque` começa em `0`, então a primeira execução já subtrai `0` sem erro nenhum, escondendo o problema até o dia em que houver um bônus de verdade rodando.

VALE PENSAR

Por que checar `if self._turnos_bonus_ataque == 0` **depois** de descontar, e não checar `if self._turnos_bonus_ataque == 1` antes? As duas formas contam o mesmo número de turnos — mas uma delas deixa `_turnos_bonus_ataque` sempre correto para quem quiser ler "quantos turnos faltam" a qualquer momento, inclusive no meio do bônus. Qual das duas é essa?

Elixir de fúria

Os dois métodos de hoje moram no `Personagem`. Falta a peça que os liga ao jogo: um item, em `itens.py`, que chama `aplicar_bonus_ataque_temporario` em vez de `melhorar_ataque`.

O QUE ELE PRECISA FAZER

- Herdar de `Item`, como todo item.
- Guardar `self.forca` (quanto de ataque) e `self.duracao` (quantos turnos).
- Em `usar(self, dono, inimigo)`, chamar `dono.aplicar_bonus_ataque_temporario(self.forca, self.duracao)`.

```
class ElixirDeFuria(Item):
    def __init__(self):
        super().__init__("Elixir de fúria")
        self.forca = 6
        self.duracao = 3

    def usar(self, dono, inimigo):
        dono.aplicar_bonus_ataque_temporario(self.forca, self.duracao)
        print(dono.nome, "bebeu o elixir de fúria")
```

QUANTO CÓDIGO ISSO É

Sete linhas — do mesmo tamanho da `PocaoDeForca` de dois dias atrás. O item continua simples porque toda a complexidade de "lembrar e desfazer" mora no `Personagem`, não no item.

Poção de força, Elixir de fúria

PocaoDeForca (permanente)

```
class PocaoDeForca(Item):
    def __init__(self):
        super().__init__(
            "Poção de força")
        self.forca = 5

    def usar(self, dono, inimigo):
        dono.ataque += self.forca
        print(dono.nome,
              "ganhou ataque")
```

ElixirDeFuria (temporário)

```
class ElixirDeFuria(Item):
    def __init__(self):
        super().__init__(
            "Elixir de fúria")
        self.forca = 6
        self.duracao = 3

    def usar(self, dono, inimigo):
        dono.aplicar_bonus_ataque_temporario(
            self.forca, self.duracao)
        print(dono.nome,
              "bebeu o elixir de fúria")
```

- ganha `duracao`
- chama um método diferente dentro de `usar`

A DIFERENÇA QUE IMPORTA

As duas classes têm a mesma forma por fora: um `__init__` que guarda números, um `usar(self, dono, inimigo)` que aplica o efeito. A diferença inteira está em **qual método do personagem** cada uma chama por dentro — `dono.ataque += self.forca` contra `dono.aplicar_bonus_ataque_temporario(...)`. O item não sabe nem precisa saber como o bônus vai ser desfeito depois; essa responsabilidade é do `Personagem`.

Testando

```
h = Guerreiro("Thoric")
ElixirDeFuria().usar(h, None)
print("ataque:", h.ataque)
```

SAÍDA

```
Thoric sente o ataque aumentar por um tempo
Thoric bebeu o elixir de fúria
ataque: 24
```

Por que o elixir dá mais que a poção

A Poção de força dá `5` de ataque, para sempre. O Elixir de fúria dá `6`, mas só por `3` turnos. Não é acidente: um efeito que expira pode, por design, valer um pouco mais por turno — ele nunca vai competir com a soma acumulada de várias poções permanentes ao longo do jogo inteiro.

VALE PENSAR

Se um dia este jogo precisasse de um item que soma `+10` de ataque só na luta atual, sem contar turnos — só "enquanto durar o combate" — o que mudaria? `aplicar_bonus_ataque_temporario` conta turnos, não lutas. Encaixar as duas ideias no mesmo método provavelmente pediria um parâmetro a mais, ou um método separado. Hoje o jogo só precisa de uma noção de "temporário", e ela é por turnos.

Vale reforçar o que **não** mudou: `abrir_mochila`, no `main.py`, continua chamando só `item.usar(jogador, inimigo)`, sem saber se o item é permanente, temporário, ou nenhuma das duas coisas. É o mesmo polimorfismo de sempre — só que agora um item pode conversar com um lado novo do personagem.

O elixir como prêmio de fase

Uma troca na lista de fases, no `main.py` — o mesmo lugar de sempre:

```
fases = [  
    Fase("Floresta Sombria", "Um Goblin salta na sua frente.",  
        Goblin(), PocaDeVida()),  
    Fase("Trilha da Montanha", "Um Orc bloqueia o caminho.",  
        Orc(), ElixirDeFuria()),  
    Fase("Ponte Quebrada", "Um Troll ronca embaixo da ponte.",  
        Troll(), AnelDeProtecao()),  
    Fase("Covil do Dragão", "O Dragão Ancião desperta.",  
        Dragao(), PocaDeVida()),  
]
```

Nada além dessa linha muda. `fase.premio` já sabia entregar qualquer item, e `abrir_mochila` já sabia usar qualquer item — os dois foram escritos antes do Elixir de fúria existir, e nenhum dos dois precisou descobrir que ele existe agora.

O turno em que o elixir é bebido

Um detalhe de ordem, dentro de `combate`:

```
jogador.iniciar_turno()  
turno_do_jogador(jogador, inimigo) # é aqui que o elixir pode ser bebido
```

`iniciar_turno` roda **antes** da escolha do jogador. Isso significa que beber o elixir num turno não desconta esse mesmo turno da contagem — o primeiro desconto só acontece no próximo `iniciar_turno`, um turno depois. É por isso que, no teste de duas páginas atrás, o ataque ficou em `24` por dois turnos inteiros antes de voltar a `18` no terceiro.

Um bônus que sabe ir embora

Nos exercícios de hoje: o mesmo padrão, pela segunda vez — a versão para defesa, sozinhos, sem o código dado. Na próxima aula: o jogo passa a ter personagens de verdade diferentes; comece a pensar no que faria um Mago realmente parecer um Mago.

- ✓ `self._bonus_ataque` e `self._turnos_bonus_ataque` existem no `__init__`, começando em `0`
- ✓ `aplicar_bonus_ataque_temporario(quantidade, turnos)` soma o ataque e guarda os dois números
- ✓ `iniciar_turno` desconta um turno por vez, e só desfaz o bônus quando a contagem chega a zero
- ✓ `ElixirDeFuria` existe em `itens.py` e chama `aplicar_bonus_ataque_temporario`, não `melhorar_ataque`
- ✓ Pelo menos uma fase dá o elixir de prêmio